International Journal of Pure and Applied Mathematics
Volume 119 No. 12 2018, 13807-13814
ISSN: 1314-3395 (on-line version)
url: http://www.ijpam.eu
Special Issue

ijpam.eu

Load rebalancing with advanced encryption standard for Hadoop Distributed File System using Distributed Hash table

M.Nithya[1], N.UmaMaheshwari[2]
[1]PG Scholar, [2]Professor, Department of Computer Science and Engineering,
PSNAcollege of Engineering and Technology,Dindigul,
nithiyanithu9589@gmail.com ,numamahi@gmail.com

Abstract-Distributed file systems are understanding something to building blocks for cloud computing practical use is based on the MapReduce programming paradigm. In distributed system, all the systems are make necessary to be a process of one common administrator and be able to secure communication. To convey safely, Advanced Encryption Standard (AES) calculation is utilized to empower the condition of being secured the information at at each clusters. It accomplish encryption and decryption in preference to read and write appropriately. Encryption and Decryption are understanding something is used for securing Hadoop Distributed File System. The existing system be controlled by a single name node to manage almost all operations of every data block in the file system. As a result it can be a bottleneck resource and a single point of failure. To overcome this failure , Load Rebalancing Algorithm is used for Hadoop Distributed File System using Distributed Hash Table . The Proposed load rebalancingalgorithm will be compared against a centralized approach in a production system and a competing distributed solution is presented in the literature. The storage nodes are give a pattern to a network based on distributed hash table become aware of a file chunk can simply refer to rapid key lookup in DHTs, given that a unique handle (or identifier) is assigned to each file chunk

I INTRODUCTION

Hadoop File System was developed using distributed file system design. It runs on commodity hardware. Unlike other distributed systems, HDFS is highly fault tolerant and designed using low-cost hardware. Offloading the load rebalancing task is suggested to have storage nodes by having the storage nodes balance their loads spontaneously. There are two main methods they are:

HDFS Pipelining
HDFS grouping

A HDFS Pipelining
In first stage, from main system, Data can be subdivided into various sub system by multiple key values using HDFS pipelining method. Files are distributed among several sub servers in priority wise i.e., HDFS pipelining method and Load Rebalancing Algorithm task also performed When files are written to HDFS a number of things are going on behind the scenes related to HDFS block consistency and replication. The main IO component of this process is by far replication. There is also the bidirectional communication with the name node registering the block's existence and state.

B HDFS Grouping
A map reduce based data archiving scheme keeps each mapper's intermediate output key value pairs in a local key value store. With the local key value store in place HDFS grouping merges all the intermediate key value pairs with the single key value pair.

C Distributed Hash Table (DHT)
The proposed system proposed the method DISTRIBUTED HASH TABLE (DHT).A distributed hash table (DHT) is a class of a decentralized distributed system that provides a lookup service similar to a hash table: (*key*, *value*) pairs are stored in a DHT, and any participating node can efficiently retrieve the value associated with a given key. Responsibility for maintaining the mapping from keys to values is distributed among the nodes, in such a way that a change in the set of participants causes a minimal amount of disruption. This allows a DHT to scale to extremely large numbers of nodes and to handle continual node arrivals, departures, and failures.

D Advanced Encryption Standard
The more popular and widely adopted symmetric encryption algorithm likely to be encountered nowadays is the Advanced Encryption Standard (AES). It is found at least six time faster than triple DES.

The schematic of AES structure is given in the following illustration −

E Encryption Process
Here, we restrict to description of a typical round of AES encryption. Each round comprise of four sub-processes.

a. Byte Substitution (SubBytes)

The 16 input bytes are substituted by looking up a fixed table (S-box) given in design. The result is in a matrix of four rows and four columns.

b. Shiftrows

Each of the four rows of the matrix is shifted to the left. Any entries that 'fall off' are re-inserted on the right side of row.

c. MixColumns

Each column of four bytes is now transformed using a special mathematical function. This function takes as input the four bytes of one column and outputs four completely new bytes, which replace the original column. The result is another new matrix consisting of 16 new bytes. It should be noted that this step is not performed in the last round.

d. Addroundkey

The 16 bytes of the matrix are now considered as 128 bits and are XORed to the 128 bits of the round key. If this is the last round then the output is the ciphertext. Otherwise, the resulting 128 bits are interpreted as 16 bytes and we begin another similar round.

F Proxy Re-Encryption

Proxy re-encryption scheme are cryptosystems which allow third parties (proxies) to alter a ciphertext which has been encrypted for one party, so that it may be decrypted by another. Proxy re-encryption schemes are similar to traditional symmetric or asymmetric encryption schemes.

II RELATED WORK

Yuanqi Chen et.al (2017) have proposed the Erasure-coded data archival system called aHDFS for Hadoop clusters, where RS (k+r, k) codes are employed to archive data replicas in the Hadoop distributed file system or HDFS. We develop two archival strategies (i.e., aHDFS-Grouping and aHDFS-Pipeline) in aHDFS to speed up the data archival process. aHDFS-Grouping - a Map Reduce -based data archiving scheme - keeps each mapper's intermediate output Key-Value pairs in a local key-value store. With the local store in place, aHDFS-Grouping merges all the intermediate key-value pairs with the same key into one single key-value pair, followed by shuffling the single Key-Value pair to reducers to generate final parity blocks. In this system, it reduces the network I/O traffic. Data reconstruction system to deal with block failure issues on Hadoop clusters. In Future, to apply the grouping and pipelining strategies will be used to the reconstruction system to speed up the reconstruction process. To optimize reconstruction performance. It is proposed to investigate a way of choosing intermediate parity blocks to be kept in the local key-value store.

Runhui Li et.al (2017)have proposed to balance performance and storage efficiency, modern clustered file systems often first store data with replication, followed by encoding the replicated data with erasure coding. Clustered file systems (CFSes) ensure data availability by striping data with redundancy across different nodes in different racks, such that any subset with a sufficient number of encoded blocks can reconstruct the original data blocks. Replication improves read performance as it cans load-balance read requests across multiple replicas. On the other hand, erasure coding improves storage efficiency with much less storage redundancy, while it achieves the same or even higher fault tolerance than replication based on reliability analysis. Maintain load balancing across replicas as in random replication before the encoding operation. Local repairable codes are a new family of erasure codes that reduce I/O during recovery while limiting the number of surviving nodes to be accessed.

Jianzhong Huang et.al (2015) have proposed a new scaling scheme called Scale-RS, which aims to expand RS-coded chunk groups for storage clusters. It is important to scale storage systems to satisfy growing demands of storage capacity and I/O performance. In this study, we propose an efficient scaling scheme—ScaleRS—for Reed-Solomon-coded storage clusters. Scale-RS aims at boosting the I/O performance of scaled storage clusters in terms of read parallelism and write throughput. Scale-RS is conducive to optimizing both data migrations and parity updates. On one hand, Scale-RS moves only necessary data blocks from old data chunks to new data chunks without migrating data among old data chunks; on the other hand, with the transposed data layout in place, Scale-RS generates associated parity difference blocks from data blocks stored in an individual data chunk, thereby substantially reducing update traffic. In future, to develop novel strategies to schedule migration I/Os among data chunks according to access locality in user I/O workload, thereby aiming to minimize interference between migration I/Os and user I/Os.

Zujie Ren et.al (2014) have presented the analysis of Hadoop trace derived from a 2,000-node production Hadoop cluster in Taobao, Inc. The trace covers the jobs execution logs over a two-week period, which common in data platform for an e-commerce web site. The Map Reduce framework, proposed by Google, provides a highly scalable solution for processing large-scale data. The fundamental concept of Map Reduce is to distribute data among many nodes and process the data in a parallel manner. Hadoop, an open-source implementation of the Map Reduce framework, can easily scale out to thousands of nodes and work with petabyte data. Due to its high scalability and performance. Our work was originally motivated by the need for improving the system performance and resource utilization of a Hadoop cluster in Taobao. It can help other researchers and engineers understand the performance and job characteristics of Hadoop in their production environments. Here the bottle neck problem will occur. It is proposed and implemented a priority-based fair scheduler aiming to optimize the efficiency of scheduling small jobs.

Jun Wang et.al (2014) Existing data parallel frameworks e.g. Hadoop, or Hadoop-based clouds, distribute the data using a random placement method for simplicity and load balance. The emerging myriad data intensive applications place a demand on high performance computing resources with massive storage. Academic and industrial pioneers have been developing big data parallel computing frameworks and large-scale distributed file systems to facilitate the high-

performance runs of data-intensive applications. In this paper, we develop a new Data-gRouping - AWare data placement scheme (DRAW) that takes into account the data grouping effects to significantly improve the performance for data-intensive applications with interest locality. Without loss of generality, DRAW is designed and implemented as a Hadoop-version prototype. For a multi-rack Hadoop cluster, DRAW is launched at rack level (inter-rack) to manage the data distribution. DRAW captures runtime data grouping patterns and distributes the grouped data as evenly as possible. Cluster many grouped data into a small number of nodes, which limits the data parallelism degree and results in performance bottleneck.

MaheswaranSathiamoorthy et.al (2013) Distributed storage systems for large clusters typically use replication to provide reliability. Distributed storage systems for large clusters typically use replication to provide reliability. Recently, erasure codes have been used to reduce the large storage overhead of three replicated systems. Reed-Solomon codes are the standard design choice and their high repair cost is often considered an unavoidable price to pay for high storage efficiency and high reliability. This paper shows how to overcome this limitation. It is proposed a novel family of erasure codes that are efficiently repairable and offer higher reliability compared to Reed Solomon codes. It show analytically that our codes are optimal on a recently identified trade off between locality and minimum distance. We implement our new codes in Hadoop HDFS and compare to a currently deployed HDFS module that uses Reed Solomon codes. Modern storage systems are transitioning to erasure coding. We introduced a new family of codes called Locally Repairable Codes (LRCs) that have marginally suboptimal storage but significantly smaller repair disk I/O and network bandwidth requirements. It believe that LRCs create a new operating point that will be practically relevant in large-scale storage systems, especially when the network bandwidth is the main performance bottleneck.

jianzhonghuang et.al (2013) have present a power-efficient scheme for erasure-coded storage clusters—ECS2—which aims to offer high energy efficiency with marginal reliability degradation. ECS2 utilizes data redundancies and deferred writes to conserve energy. In ECS2 parity blocks are buffered exclusively in active data nodes whereas parity nodes are placed into low-power mode The goal of this paper is to quantify service latency under these two policies and to develop an optimization framework to minimize differentiated service latency for multiple tenants. Using the latency analysis, It proposed a novel optimization framework for minimizing differentiated service latency. That existing energy conservation techniques for storage clusters reduce energy consumption at the cost of performance or reliability of the systems.

Hung-Chang Hsiao et.al (2012) In this paper, a fully distributed load rebalancing algorithm is presented to cope with the load imbalance problem. Distributed file systems are key building blocks for cloud computing applications based on the Map Reduce programming paradigm. In such file systems, nodes simultaneously serve computing and storage functions; a file is partitioned into a number of chunks allocated in distinct nodes so that Map Reduce tasks can be performed in parallel over the nodes. In such a distributed file system, the load of a node is typically proportional to the number of file chunks the node possesses. The computer simulation results are encouraging, indicating that our proposed algorithm performs very well. Our proposal is comparable to the centralized algorithm in the Hadoop HDFS production system and dramatically outperforms the competing distributed algorithm. It provide physical network locality and node heterogeneity. The limitation in the number of storage nodes, the number of files and the number of accesses to files increase linearly, the central nodes become a performance bottleneck.

O. Khan et.al (2012) Existing solutions to balance load in DHTs incur a high overhead either in terms of routing state or in terms of load movement generated by nodes arriving or departing the system. To reduce storage overhead, cloud file systems are transitioning from replication to erasure codes. This process has revealed new dimensions on which to evaluate the performance of different coding schemes: the amount of data used in recovery and when performing degraded reads. We present an algorithm that finds the optimal number of codeword symbols needed for recovery for any XOR-based erasure code and produces recovery schedules that use a minimum amount of data. Cloud file systems transform the requirements for erasure codes because they have properties and workloads that differ from traditional file systems and storage arrays. Increasing the symbol size has drawbacks as well. It increases memory consumption during recovery and increases the latency of degraded reads, because larger symbols need to recover more data. It provides load balance and incremental scalability in the data center. Several open problems remain with respect to optimal recovery and degraded reads. While our algorithm can determine the minimum number of symbols needed for recovery for any given code, it remains unknown how to generate recovery-optimal erasure codes

D. Ford et.al (2010)A fundamental problem that confronts peer-to-peer applications is the efficient location of the node that stores a desired data item. Highly available cloud storage is often implemented with complex, multi-tiered distributed systems built on top of clusters of commodity servers and disk drives. Cloud storage is often implemented by complex multi tiered distributed systems on clusters of thousands of commodity servers. It find that correlation among node failures dwarfs all other contributions to unavailability in our production environment. In particular, though disks failures can result in permanent data loss, the multitude of transitory node failures account for most unavailability. We present a simple time-window-based method to group failure events into failure bursts which, despite its simplicity, successfully identifies bursts with a common cause. We develop analytical models to reason about past and future availability in our cells, including the effects of different choices of replication, data placement and system parameters. This paper presents Chord.

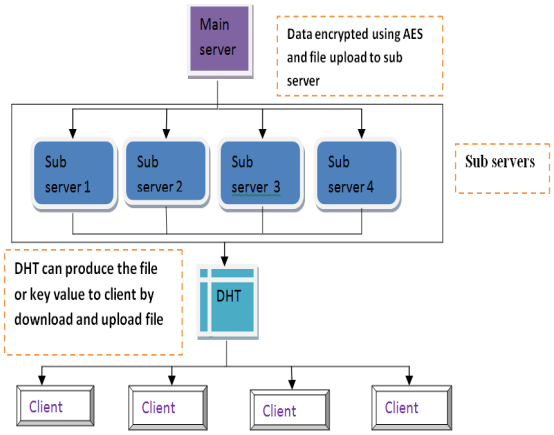
III PROPOSED MODEL

The proposed system works in two phases. In phase I, the chunk creation, DHT formulation and Resource allocation will done and the client will upload the data with AES Encryption. Then in phase II inside the cloud again the data has divided into small pieces, for this process we will apply a dividing key and the replica management and data retrieval can be maintained. In a proposed system, a fully

distributed load rebalancing algorithm is presented to cope with the load imbalance problem. Moreover, as failure is the norm, nodes are newly added to sustain the overall system performance resulting in the heterogeneity of nodes. This eliminates the dependence on central nodes. The storage nodes are structured as a network based on distributed hash tables. Our algorithm is compared against a centralized approach in a production system and a competing distributed solution presented in the literature. DHTs enable nodes to self-organize and repair while constantly offering lookup functionality in node dynamism, simplifying the system provision and management. The objective of this proposed system is to reduce network traffic or movement cost caused by rebalancing the loads of nodes as much as possible to maximize the network bandwidth available to normal applications. We construct a secure cloud storage system that supports the function of secure data forwarding by using an AES and Proxy re encryption. In this model initial phase owner will upload the data with AES Encryption. Next phase, inside of cloud again the data has divided into small pieces, for this process we will apply a dividing key. Then in order to detect the block failure, the data reconstruction is done using load rebalancing algorithm.

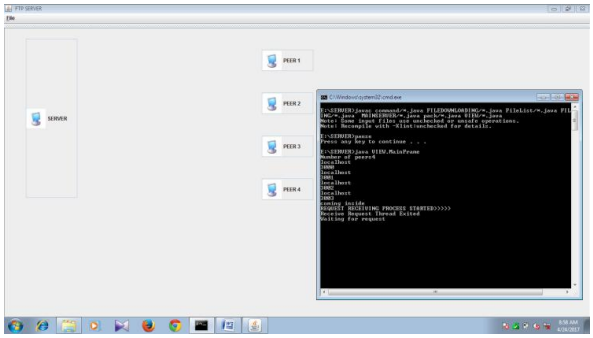
IV ARCHITECTURE OF LOAD REBALANCING FOR HDFS USING DHT

The proposed architecture of load rebalancing for HDFS using DHT is shown in Fig.3.1. The various steps of the proposed work are explained in this subsection.In first stage, from main system, Data can be subdivided into various sub system by multiple key values using HDFS pipelining method. Files are distributed among several sub servers in priority wise i.e., HDFS pipelining method and Load Rebalancing Algorithm task also performed. To communicate securely, the proposed Advanced Encryption Standard (AES) algorithm is used to enables protection of data at each cluster. It performs the encryption and decryption before read and write respectively. Encryption and Decryption in key is used for securing Hadoop Distributed File System. DHTs enable nodes to self-organize and -repair while constantly offering lookup functionality in node dynamism, simplifying the system provision and management. Finally, Centralized system I.e., Distributed Hash Table can produce the key value to client. HDFS grouping method is used to download and upload the data in Distributed Hash Table in the final stage.

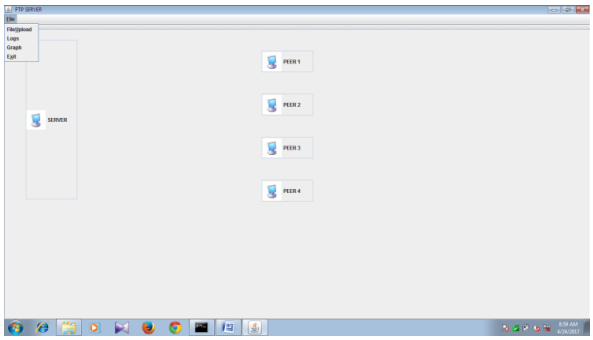


V EXPERIMENTAL RESULT

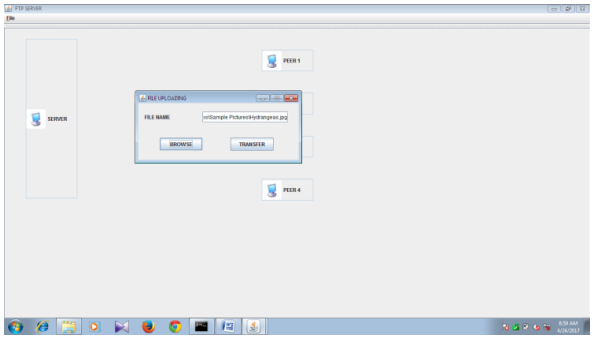
Step 1: Chunk creation



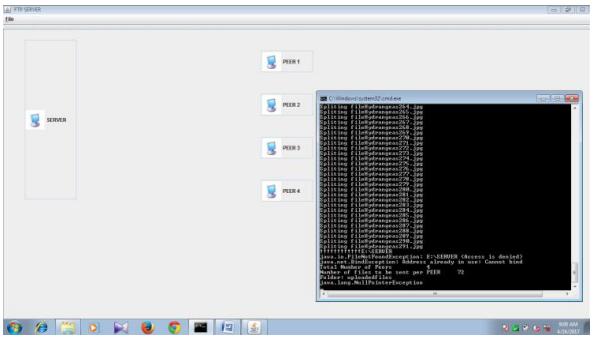
Step 2: File Upload to every peer



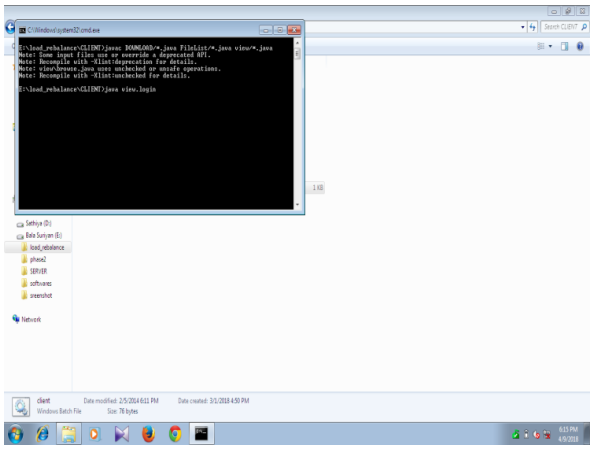
Step 3: Upload file from system or browse the file

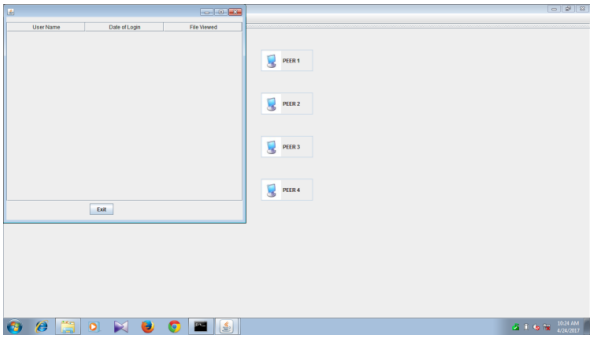


Step 4: Main server can be upload the file automatically and load balanced to each and every peer servers

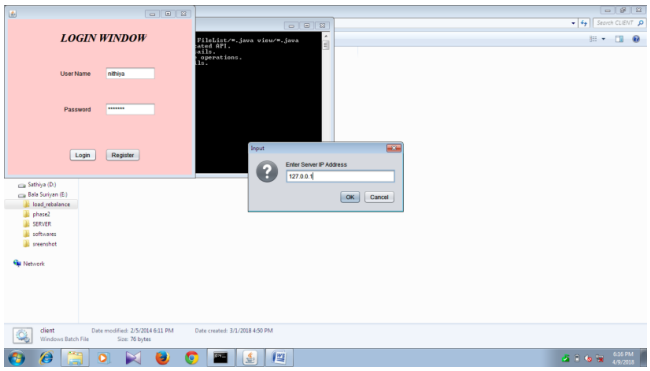


Step 5: Details about all users who are all access the servers

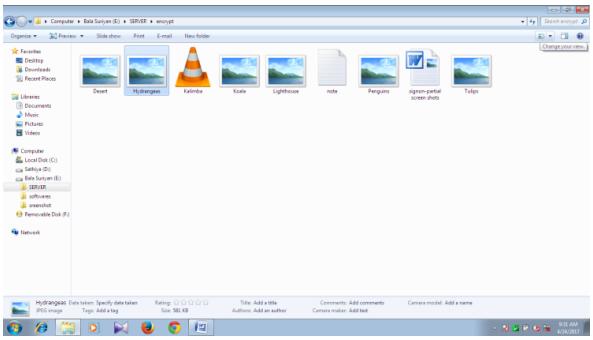




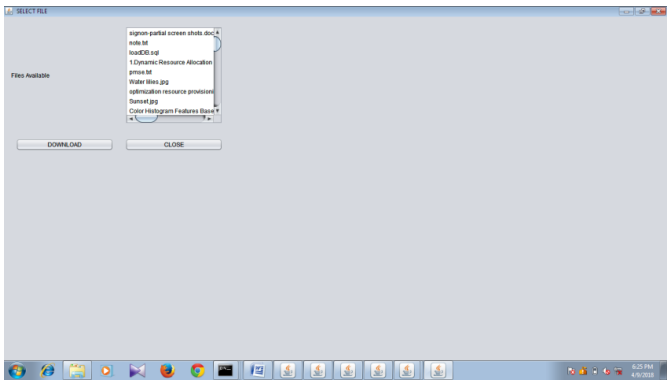
Step 6: All files are uploaded to peer servers are stored here

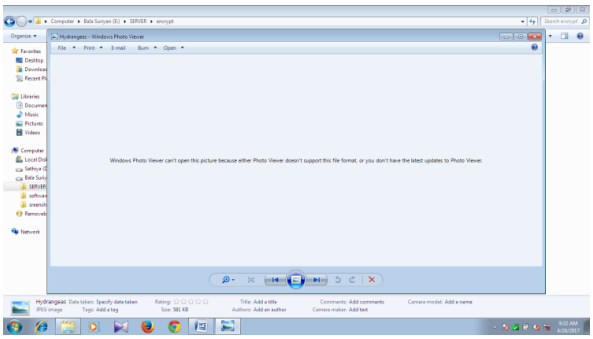


Step 9: Download the File from Distributed Hash Table

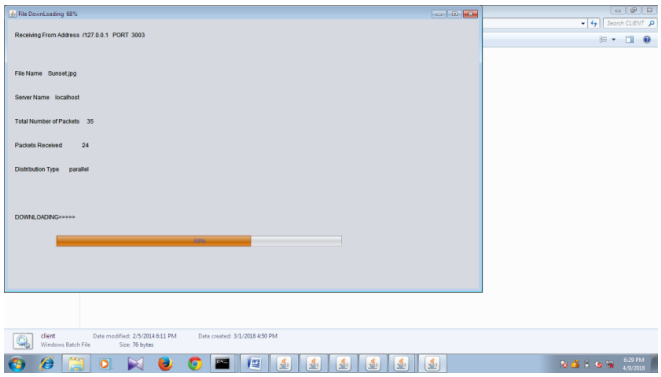


Step 7: All files upload here are encrypted with AES

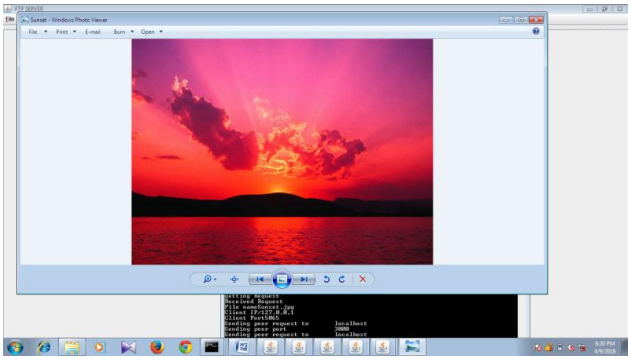




Step 8: open the Client Login



Step 10: View the original File by Client After authenticated



VI CONCLUSION

Erasure codes are promising for improving the reliability of the storage system due to its space efficiency compared to the replication methods. Traditional erasure codes split data into equalsized data blocks and encode strips in different data blocks. This brings heavy repairing traffic when clients read parts of the data, since most strips read for repairing are not in the expected blocks. This paper proposes a novel discrete data dividing method to completely avoid this problem. The key idea is to encode strips from the same data block. We could see that for repairing failed blocks, the strips to be read are either in the same data block with corrupted strips or from the encoded strips. Therefore, no data is wasted. We design and implement this data layout into a HDFS-like storage system. Experiments over a small-scale test bed shows that the proposed discrete data divided method avoids downloading data blocks that are not needed for clients during the repairing operations.

VIIFUTURE WORK

As a response, erasure coding as an alternative to backup has emerged as a method of protecting against drive failure. Raid just does not cut it in the age of high-capacity HDDs. The larger a disk's capacity, the greater the chance of bit error .And when a disk fails, the Raid rebuild process begins, during which there is no protection againsta second (or third) mechanism failure. So not only has therisk of failure during normal operation grown with capacity, it is much higher during Raid rebuild, too. Also, rebuild times were once measured in minutes or hours, but disk transfer rates have not kept pace with the rate of disk capacity expansion, so large Raid rebuilds can now take days or even longer.

REFERENCES

[1] D. Borthakur, “The hadoop distributed file system: Architecture and design,” Hadoop Apache Project. http://hadoop.apache.org/common/docs/current/hdfs design.html, 2009.

[2] Dean .J and Ghemawat . S, “Mapreduce: simplified data processing on large clusters,” Communications of the ACM, vol. 51, no. 1, p p. 107–113, 2008.

[3] Dimakis, A., Prabhakaran, V., Ramchandran, K.: Decentralized erasure codes for distributed networked storage. IEEE/ACM Transactions on Networking 14 (2006)

[4] D. Ford, F. Labelle, F. Popovici, M. Stokely, V. Truong, L. Barroso, C. Grimes, and S. Quinlan, “Availability in globally distributed storage systems,” in Proc. 9th USENIX Symp. Operating Syst. Des. Implementation, 2010, pp. 1–14.

[5] S. Ghemawat., H. Gobioff., and S.-T. Leung., “The Google File
System,”in ACM SIGOPS operating system review, vol.37,no.5. ACM Journal,2003, pp. 29-43.

[6] C. Guo, H. Wu, K. Tan, L. Shi, Y. Zhang, and S. Lu. DCell: a scalable and fault-tolerant network structure for data centers. SIGCOMM Comput. Commun. Rev., 38:75–86, August 2008

[7] R. Gupta, H. Gupta, U. Nambiar, and M. Mohania, “Efficiently querying archived data using hadoop,” in Proceedings of the 19th ACM international conference on Information and knowledge management. ACM, 2010,pp. 1301–1304.

[8] HDFS-RAID. http://wiki.apache.org/hadoop/HDFS-RAID.

[9] J. Huang., X. Liang., X. Qin., P. Xie., and C. Xie., “Scale-rs: An efficient scaling scheme for rs-coded storage clusters,” Parallel and Distributed Systems, IEEE Transactions on, vol. 26, no. 6, pp. 1704–1717, 2015

[10] J. Huang., Y. Wang., X. Qin., X. Liang., S. Yin., and C. Xie., “Exploiting pipelined encoding process to boost erasure-coded data archival.,” Parallel and Distributed Systems, IEEE Transactions on, vol. 26, no. 11, pp. 2984–2996, 2015

[11] Hung-Chang Hsiao., Hsueh-Yi Chung., Haiying Shen., Yu-Chang chao.,“Load Rebalancing for distributed file system in clouds” IEEE Journal of Transactions on parallel and distributed systems, volume:24,Issue.5,pp. 951-962, 2013

[12] Jun Wang., Pengju Shang., and Jiangling Yin., ““DRAW: A New Data-gRouping-AWare Data Placement Scheme for Data Intensive Applications With Interest Locality”IEEE Journals of Transactions on Magnetics ,Volume:49,Issue:6, June 2013

[13] Kaushik .R.T and Nahrstedt .K, “T: a data-centric cooling energy costs reduction approach for big data analytics cloud,” in Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis. IEEE Computer Society Press, p.52, 2012,

[14] O. Khan, R. Burns, J. Plank, W. Pierce, and C. Huang. Rethinking erasure codes for cloud file systems: minimizing

I/O for recovery and degraded reads. In FAST, 2012.

[15] E. Medernach, "Workload analysis of a cluster in a grid environment," in Job Scheduling Strategies for Parallel Processing, 2005, pp. 36–61.

[17] J. Mickens and B. Noble. Exploiting availability prediction in distributed systems. In NSDI, 2006.

[18] L. Pamies-Juarez, A. Datta, and F. Oggier, "Rapidraid: Pipelined erasure codes for fast data archival in distributed storage systems," in INFOCOM, 2013 Proceedings IEEE. IEEE, 2013, pp. 1294–1302.

[19]L. Pamies-Juarez, F. Oggier, and A. Datta, "Decentralized erasure coding for efficient data archival in distributed storage systems," in Distributed Computing and Networking. Springer, 2013, pp. 42–56.

[20] D. Papailiopoulos and A. G. Dimakis. Locally repairable codes. In ISIT 2012.

[21] D. Papailiopoulos, A. Dimakis, and V. Cadambe. Repair optimal erasure codes through hadamard designs. IEEE Trans. Inf. Th., May 2013.

[22] K. V. Rashmi, N. B. Shah, D. Gu, H. Kuang, D. Borthakur, and K. Ramchandran. A solution to the network challenges of data recovery in erasure-coded distributed storage systems: A study on the Facebook warehouse cluster. In Proc. USENIX HotStorage, June 2013.

[23] K. Rashmi, N. B. Shah, D. Gu, H. Kuang, D. Borthakur, and K. Ramchandran, "A hitchhiker's guide to fast and efficient data reconstruction in erasure-coded data centers," in Proceedings of the 2014 ACM conference on SIGCOMM, pp. 331–342, ACM, 2014.

[24] I. S. Reed and G. Solomon, "Polynomial codes over certain finite fields," Journal of the society for industrial and applied mathematics, vol. 8, no. 2, pp. 300–304, 1960.

[25] R. Rodrigues and B. Liskov. High availability in DHTs: Erasure coding vs. replication. In IPTPS, 2005.